Digital Integrated Systems

Bachelor Thesis

Design method of digital circuits with LLMs

DeepSpec: Specification-Enhanced Circuit Generation Using LLMs

Supervisor: Prof. Bing Li

Author: Zimeng Wu

September 14, 2025

Abstract

Large Language Models (LLMs) have been used for automatic RTL design in digital circuits recently. However, the inherent instability and imprecision of LLMs often lead to ambiguous natural language descriptions during text augmentation and functional errors in code generation, ultimately causing task failures. Previous approaches have either focused on time-consuming general full-stack workflows that lack attention to specific detail, or have heavily relied on increasingly sophisticated reasoning models for back-end code generation and correction, particularly for complex sequential circuits. To address these issues, we propose DeepSpec as a preprocessing stage, which not only generates a highly human-readable specification but also performs atomic information decomposition and similarity analysis, outputting potential specification conflicts that could cause code generation errors. Furthermore, to address the complexity of sequential circuits, DeepSpec first analyses pure combinational logic, then treats it as a black box for pure sequential processing, and finally combines them together. Experimental results on the VerilogEval-Human benchmark demonstrate a substantial average improvement in functional correctness (pass@1) in multiple LLMs.

Contents

1 Introduction **3**

2 Background and Motivation **5**

2.1 LLM Application in RTL Design 5

2.2 Limitations of existing researches 6

3 Methodology **7**

3.1 Proposed Framework 7

4 Experimental Results **13**

4.1 Experimental Setup 13

4.2 Overall Performance Analysis 15

4.3 Comparison between Baseline and LLMgen 16

4.4 Analysis by Circuit Type 17

4.5 Ablation Study 18

5 Conclusion **21**

1 Introduction

Large Language Models (LLMs) have demonstrated significant potential and achieved notable advances in software code generation, particularly for widely used programming languages such as Python and C [1–3], which benefit from their broad applicability and extensive training datasets. In contrast, conventional hardware description language workflows continue to be labor intensive and prone to errors. Engineers must invest considerable time and effort in designing and debugging Register Transfer Level (RTL) code and testbenches, creating an increasing need for more efficient and automated approaches.

With the advancement of LLMs, their application to hardware design has attracted growing research attention. Previous studies [4], [5], [6], [7] have shown promising results in RTL code generation, while other research efforts have investigated the automation of testbench design [8, 9], RTL Code Summarization and Question Answering [10], revealing substantial application potential. The VerilogEval-Human benchmark [11], released in 2023, includes 156 common design tasks such as bit-parallel adders and shift registers. It has been widely used to assess the effectiveness of automated hardware design. Based on this, LLM performance has shown impressive improvements in RTL design: the pass rate for GPT-4 at its first release was 27%. By September 2024, GPT-4 Turbo 2024-04-09 [4] had increased this to 54.2%, while OriGen [4] achieved a pass rate of 54.4%. In December 2024, MAGE [12] on Claude 3.5 Sonnet 2024-10-22 even reached an impressive 94.8% pass rate. In January 2025, VRank [13] showed a 57.4% pass rate on GPT-4o.

However, LLMs still face significant challenges due to inherent instability, laziness [14], hallucination [15, 16], and data scarcity. These limitations severely impact their performance in content generation. Furthermore, the code LLMs generate, even when it passes tests, often lacks readability, making it difficult for human engineers to understand and debug in real-world engineering contexts. Despite this, current RTL design automation frame-

works are often greedy, focusing too much on direct end-to-end generation from natural language descriptions to RTL code. This approach overlooks the crucial need for task-specific details and contextual information, leading LLMs to misunderstandings. Moreover, methods that treat combinational and sequential circuits uniformly, ignoring their fundamental differences in complexity, are also expected to be improved.

To address these challenges and improve both the quality and applicability of hardware design automation, we propose DeepSpec, a novel specification-enhanced framework for automated circuit generation. This framework can also be layered on top of other workflows to preprocess specifications. The outline of DeepSpec is illustrated in Fig. 1. Our framework provides a comprehensive and systematic workflow that leverages the probabilistic nature of LLMs. It focuses on task-specific specification generation and its evaluation. Our contributions are summarized as follows:

- We propose DeepSpec, an automated, specification-enhanced and self-reflected workflow for circuit design using LLMs, which can be flexibly layered on top of any backend RTL code generation framework. It decouples the generation of high-quality specifications from the actual code implementation, giving our framework broad applicability.
- A multi-layered handling method is introduced to perform analyses on digital circuits. For sequential circuits in particular, this approach involves first refining the original task description [11] and then combining two separate black-box analyses: one for combinational logic functions and one for timing paths and port behavior. The results from both analyses are then integrated to form a complete summary.
- We also integrate Deep Research and a Self-Reflection mechanism to enhance the accuracy of generated specifications using our method. This is achieved by leveraging web searches to enrich factual contextual information, thereby mitigating hallucination. The self-reflection

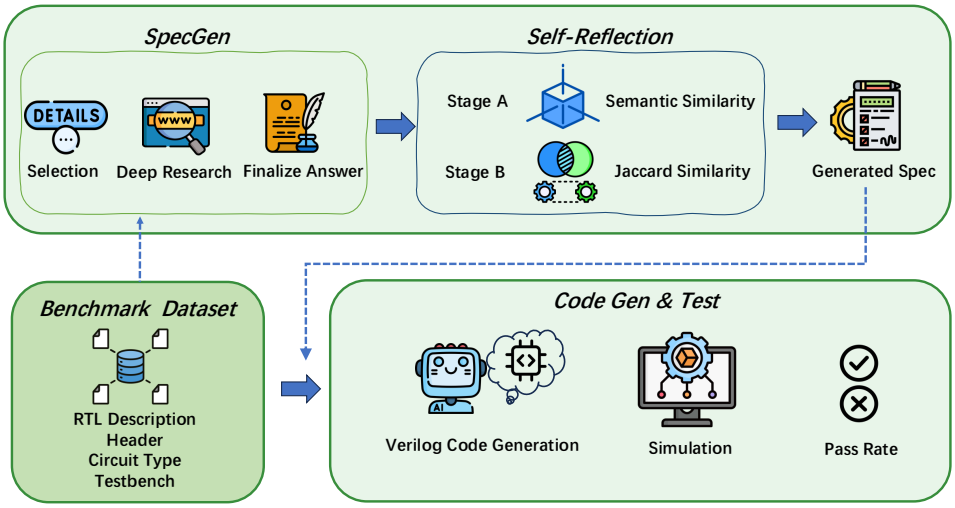


Figure 1: **Outline of DeepSpec workflow.**

process uses semantic and Jaccard similarity to ensure that the core function requirements are maintained and to evaluate the reliability of the generated specifications.

- This approach improves human-machine collaboration by generating a highly human-readable specification and an intermediate analysis that provides actionable debugging insights.

The rest of the paper is organized as follows. In Section 2, the background and motivation of this work is explained. In Section 3, we describe each component of our framework in detail. Experimental results and conclusions are shown in Section 4 and Section 5.

2 Background and Motivation

2.1 LLM Application in RTL Design

Recent works such as VRank [13], MAGE [12], and Spec2Doc2RTL [17] have made significant strides in automated RTL code generation with Large Language Models.

Leveraging different backend strategies, VRank and MAGE both enhance the quality of LLM-generated code. VRank generates multiple code candidates and selects the best one, improving functional correctness by 10.5%. MAGE, a multi-agent system, achieves a remarkable 95.7% accuracy by employing a high-temperature RTL candidate sampling system and a Verilog-state checkpoint mechanism for targeted fixes using golden testbench feedback. However, these methods rely on the assumption of a perfect "golden" testbench. In reality, such golden testbenches are often incomplete or flawed, particularly for complex designs. This means the frameworks may appear to be generating correct code, but are in fact only passing a limited set of tests, leaving critical bugs undetected and undermining the true reliability of the generated code. The limited size of common datasets also increases the risk of model memorization.

While MAGE and VRank are backend solutions, other works have focused on using natural language as a core part of the design flow. Spec2Doc2RTL, sharing a similar goal with our work, employs a hierarchical approach using natural language as an intermediate representation to decompose complex designs. This full-stack *specification-to-document-to-code* pipeline improves code generation, but the inherent lack of transparency in the generated code can make it difficult for human engineers to trust or debug. Our work takes a different approach. We introduce a decoupled framework that first enhances the initial benchmark description. It then generates a technical specification and self-reflects on it, producing debug guidance that is readily understandable by human engineers.

2.2 Limitations of existing researches

Insufficient Handling of Sequential Circuits In recent studies, a significant limitation is the insufficient handling of sequential circuits. Current methods often apply a uniform approach to both combinational and sequential circuits. However, this fails to account for the heightened complexity of se-

quential designs, which involves intricate timing, state transitions, and synchronous behavior. Consequently, these methods often result in lower success rates for sequential circuits compared with their combinational counterparts.

LLM Hallucination LLMs are prone to generating factually incorrect information or hallucinating technical details due to their training methods [18]. This is often a result of their training methods, which tend to reward confident guesses over honest "I don't know" responses. Furthermore, due to the scarcity of training resources in hardware design, the rate of incorrect guesses is without doubt higher than in software engineering.

Retrieval-Augmented Generation Many frameworks introduce RAG, Retrieval-Augmented Generation, to mitigate the impact of hallucination. While this method is proven to enable more accurate content generation, it requires significant preparation, such as searching for relational resources and chunking text. Therefore, this process is often time-consuming and labor-intensive and may not always perform as well as expected.

3 Methodology

In this section, the methodology for our specification-enhanced circuit generation framework is explained in detail, which tackles the existing limitations mentioned in Section 2 step by step.

3.1 Proposed Framework

To deal with these problems, we propose DeepSpec, a prep-processing stage for circuit generation using LLMs as depicted in Fig. 2. Three parts are explained in the following paragraphs: SpecGen, Self-Reflection and Code-Gen&Testing.

SpecGen Existing works on LLM-based RTL design primarily follow an end-to-end paradigm, directly converting natural language descriptions into

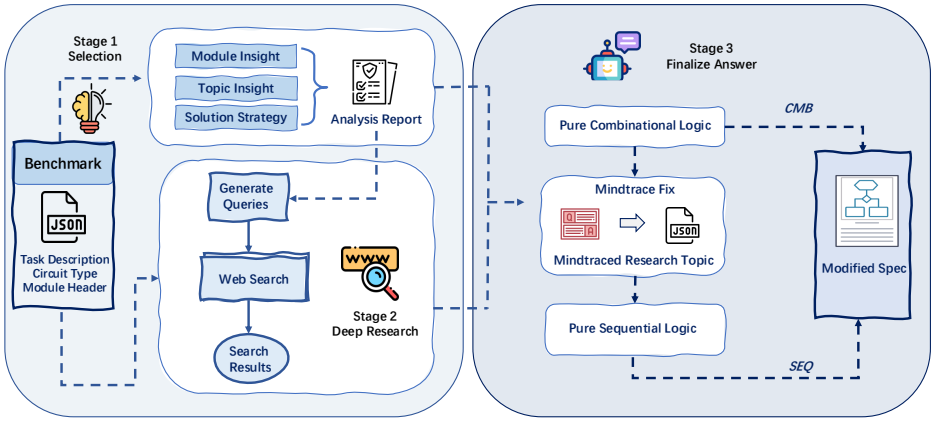


Figure 2: **Framework of SpecGen.**

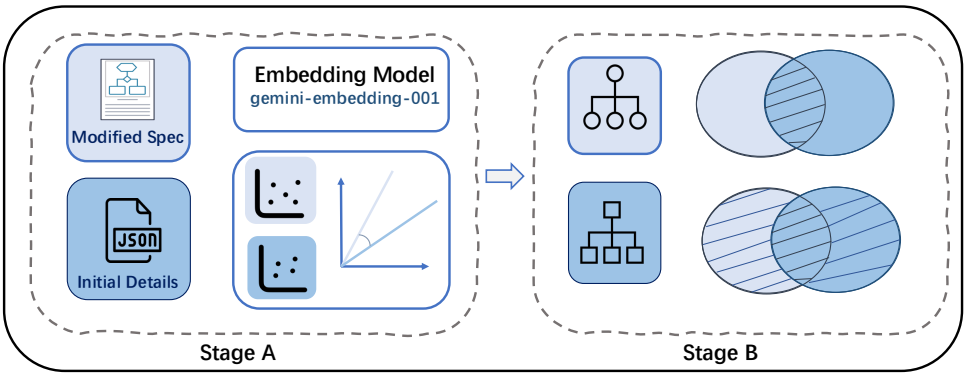


Figure 3: **Framework of Self-Reflection.**

RTL code. This approach, while seemingly efficient, overlooks crucial front-end processing. As a result, early-stage ambiguities and incompleteness can propagate throughout the entire design workflow. To address this limitation, we propose DeepSpec, a front-end framework designed to strengthen and verify circuit specifications before code generation. By ensuring correctness at the specification level, DeepSpec not only provides a reliable foundation for subsequent RTL generation but also seamlessly integrates with diverse back-end frameworks, ultimately improving overall design accuracy.

A closer look at natural language reveals its fundamental flaws. Specifica-

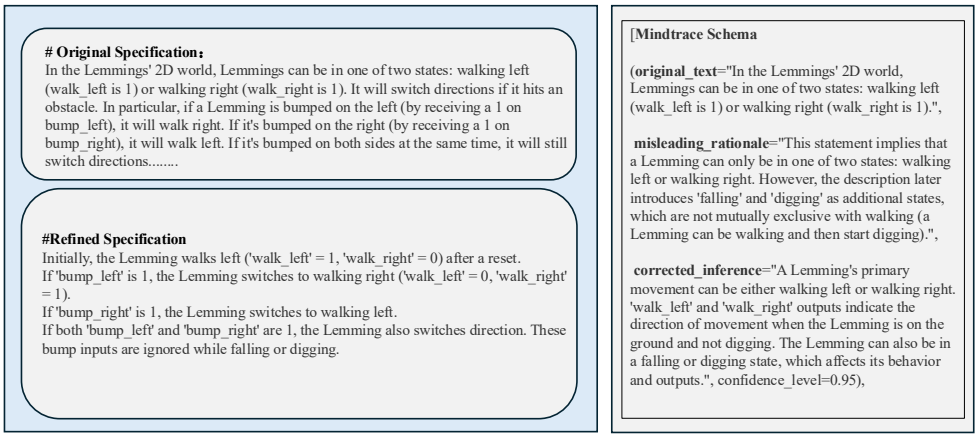


Figure 4: **Semantic Refinement**

tions are plagued by two fundamental deficiencies: *semantic incompleteness* and *structural incompleteness*. These issues critically undermine the reliability of LLM-based RTL design. Semantic incompleteness refers to the inherent ambiguity of individual sentences, while structural incompleteness occurs when sentences or paragraphs fail to form a coherent and complete description of circuit functionality. DeepSpec is a framework specifically designed to systematically address both deficiencies, generating an enhanced and trustworthy specification prior to code generation.

A major challenge in natural language specifications is semantic incompleteness: individual sentences often remain underspecified or ambiguous due to the abstraction inherent in human-written descriptions. To address this, DeepSpec employs an iterative refinement mechanism where the original specification is fed into LLMs. Ambiguous sentences are automatically identified, corrected, and annotated with reasoning, revision guidance, and confidence scores. While these intermediate outputs are not forced to be shown to the user, the framework is designed to force their generation internally. This step serves to reduce hallucination and improve the overall reliability of the output. Through multiple refinement rounds, the specification is transformed into a precise and interpretable version that eliminates sentence-level ambiguity.

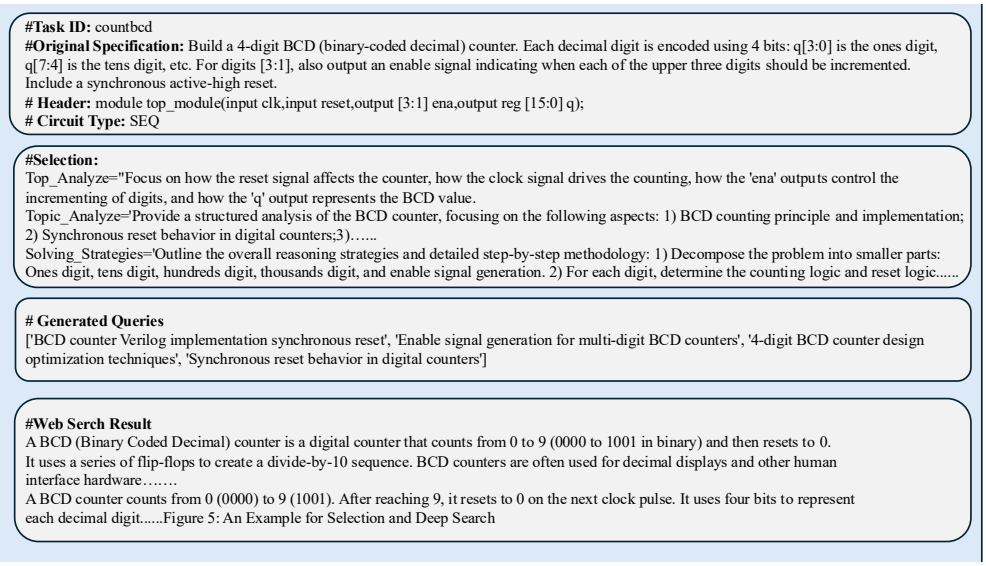


Figure 5: **An Example for Selection and Deep Research**

Beyond sentence-level issues, specifications often suffer from structural incompleteness, where paragraphs lack coherence and logical dependencies across sentences remain implicit. We attribute this problem to three core sources: (i) *Knowledge Gaps*, which refers to missing universal or circuit-specific background information; (ii) *Logic Gaps*, which is an inability to organize reasoning about circuit behavior particularly with mixed combinational and sequential information; and (iii) *Solution Gaps*, which is the absence of a high-level strategy needed to bridge the other two gaps. DeepSpec tackles these issues through a multi-layered approach.

To address (i) *Knowledge Gaps*, DeepSpec employs targeted web searches rather than retrieval-augmented generation. It generates task-specific queries by integrating insights from the original specification with an analysis of its module headers, functions, and overall strategy, respectively referred to as module insight, topic insight, and solution strategy, as illustrated in Fig. 2. The aggregated search results are organized into a knowledge base at *Stage 2* Deep Research Module, providing the LLM with the factual grounding needed to fill both universal and circuit-specific knowledge gaps. This approach is more effective because web search provides timely, lightweight, and

relevant information. An example for Selection Module and Deep Research Module is shown in Fig. 5

For (ii) *Logic Gaps*, we adopt a dual black-box abstraction: First, combinational logic is abstracted and refined, focusing on enhancing its functional description and possible implementations, then the logic blocks are treated as black boxes and analyzed for timing paths, state transitions, and interface behaviors. Next, sequential logic is focused, with emphasis on timing paths, state transitions, and port behaviors. The strong emphasis on port behavior feedback is particularly effective for handling state machines, which are highly dependent on these behaviors. Ultimately, the two layers of black-box summaries are then combined into a coherent representation of circuit functionality as shown at *stage 3* Finalizing Answer in Fig. 2.

To deal with (iii) *Solution Gaps* at the top level, DeepSpec employs a Selection Module, as shown in *Stage 1* in Fig. 2. It is a component specifically designed to bridge knowledge gaps and logic gaps, which analyzes module headers and logic functions and generates solution strategies that provide guidance on how knowledge and logic gaps should be resolved, thus ensuring global consistency of the refined specification.

The combination of knowledge gap resolution, logic gap handling, and solution gap strategies yields a structurally coherent specification, which we refer to as the enhanced specification. At the SpecGen stage, a clear and readable technical specification is produced. Since LLM outputs are inherently unstable and prone to hallucination, the correctness of refined specifications must be evaluated. DeepSpec introduces a self-reflection mechanism that validates enhanced specifications against the original description in two tiers.

Self-Reflection First, the original and enhanced specifications are converted into vectors, and semantic similarity is measured via cosine similarity of embeddings, ensuring global semantic preservation as shown in *Stage A* of Fig. 3. Second, a fine-grained Jaccard similarity is computed by decomposing

both specifications into atomic information units. Intersection and union sets of these atomic units are recorded in a human-readable JSON file, which not only provides a reliability score but also enables engineers to rapidly locate inconsistencies and receive LLM-guided debugging feedback. This human-assistive validation guarantees both semantic fidelity and detail preservation.

Second, for sequential circuits, we use *Tier 2 Jaccard Similarity* due to their complexity. The value is computed by first converting both specifications into a set of atomic, unambiguous technical requirements saved as JSON units. We then compare these two sets of units and calculate the Jaccard similarity using the formula:

$$\text{Jaccard Similarity} = \frac{\text{len(Intersection Units)}}{\text{len(Union Units)}}$$

The intersection and union sets of these units are recorded in a human-readable JSON file. This file not only provides a reliability score but also empowers engineers to rapidly locate inconsistencies and receive LLM-guided debugging feedback. This human-assistive validation guarantees both semantic fidelity and detail preservation.

CodeGen and Testing Finally, the specification is used with various LLMs to generate code, with a direct LLM code generation workflow or a state-of-the-art back-end code generation framework integrated. The resulting code is then validated against golden testbenches from the dataset.

The final stage of our methodology involves generating Verilog code and validating it through simulation tests with golden testbenches. We consider four options for the code generation phase: **Baseline:** Directly using the initial details from the benchmark as prompts for the LLM to generate code. **DeepSpec:** First generating a specification with DeepSpec, then using it as a prompt to produce code and evaluate the pass rate. **VRank [13]:** Applying VRank, a framework for multi-sample generation, cluster ranking, and candidate selection, as an integrable back-end code generation method to produce

RTL code and record simulation results. **DeepSpec + VRank:** Extending the DeepSpec workflow by integrating VRank as a full-stack framework to generate code and perform simulations. By comparing the pass rates of baseline, VRank with our front-end, we can demonstrate how our decoupled framework significantly improves the reliability and accuracy of RTL generation.

The whole workflow is state-driven and built with LangGraph [19], drawing inspiration from the gemini-fullstack-langgraph-quickstart GitHub [20]. This architecture allows the framework to manage the flow of information and decisions between each of the three stages in a structured manner.

4 Experimental Results

In this section, we present a comprehensive evaluation of the proposed DeepSpec framework on the VerilogEval-Human benchmark [11]. Our experiments are designed to progressively evaluate DeepSpec under different perspectives.

4.1 Experimental Setup

Software Environment: The framework was built with Python 3.12 which drove the optimal specification generation automation workflow and ICARUS Verilog which conducts the Verilog code simulation test. As mentioned in Section 3, Langgraph [19]was introduced which ensured smooth delivery of intermediate outputs between nodes, particularly for the implementation of stage 2 deep research from SpecGen which referred gemini-fullstack-quickstart github respository.

LLMs Selection: To demonstrate DeepSpec's generalization, we chose four representative non-inference models from both Google and OpenAI, *gemini-2.0-flash*, *gemini-2.0-flash-lite*, *gpt-4o-2024-08-06*, *gpt-4o-mini-2024-07-18* as

shown in first column of Table 1. In our work, the LLM models are used in five stages: selection, query generation, web search, answer finalization, and self-reflection. Theoretically, when testing DeepSpec's performance with a specific LLM, all five models should be kept same to conduct a full-stack workflow and to present the performance results for that LLM clearly. However, the fact is that web search capabilities are version-limited in some certain models and the API call method for vectors embedding also differs from those used for chat, we consistently used *gemini-2.0-flash google search* for stage 2 deep research and *gemini-embedding-001* for the self-reflection stage A throughout our experiments for a fair comparison, preventing the potential differencs in tool calling among these closed-source LLMs from affecting the entire workflow.

Dataset: We used the VerilogEval-Human benchmark [11], which contains 156 tasks. Each task consists of a task description, RTL module header, and golden testbenches. Following Github Respository mentioned in [8], we also leverage a supplementary circuit-type classification file to distinguish between combinational and sequential tasks.

Metrics: Three key evaluation metrics are used across the Self-Reflection and CodeGen and Testing of the DeepSpec workflow. First, semantic similarity [21] is calculated in stage A of Self-Reflection, and then Jaccard similarity [22] is used in stage B. These two metrics are crucial for determining if a modified answer can be accepted as the final technical specification. The third metric, pass@k [23], estimates the probability that at least one of the top-k generated candidates passes the golden testbench, which serves as a measure of functional correctness. The calculation formula is shown below:

$$\text{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

where n is the total number of candidates, c is the number of correct implementations, and k is the sample size. In this work pass@1 is mainly used. First, we conducted a study on how DeepSpec performs across various large

Table 1: **Performance of DeepSpec Workflow Across Various Large Language Models at pass@1.**

Model	**Baseline**		**DeepSpec**		**Increase**
	Ratio	*#Tasks*	*Ratio*	*#Tasks*	*Ratio*
gemini-2.0-flash	58.97%	92/156	67.95%	106/156	+8.98%
gemini-2.0-flash-lite	57.69%	90/156	62.18%	97/156	+4.49%
gpt-4o-2024-08-06	64.74%	101/156	67.31%	105/156	+2.57%
gpt-4o-mini-2024-07-18	53.21%	83/156	56.41%	88/156	+3.20%

language models (LLMs), with the results presented in Table 1.

4.2 Overall Performance Analysis

Introduction of Comparison Frameworks The Baseline framework generated Verilog code directly from the original task specification provided by the benchmark. In contrast, DeepSpec enhanced the specifications before code generation, aiming to reduce ambiguity and improve functional correctness and then the enhanced technical specifications were leveraged to generate code.

Results and Analysis The results are summarized in Table 1. The table's first column lists the four selected LLMs. The next five columns detail the pass@1 results for the performance comparison between Baseline and our DeepSpec. The final five columns evaluated the additive effect of our specification-enhanced framework. For each framework, we report the number of passed tasks as *#Tasks* and the success rate as *Ratio*.

As shown in Table 1, DeepSpec consistently outperformed the Baseline across all four tested language models. Specifically, *gemini-2.0-flash* achieved the largest improvement, with nearly a 9% increase in functional pass rate at pass@1. *gemini-2.0-flash-lite* also benefited significantly, with a gain of

Table 2: **Performance Comparison Between Baseline and LLMgen at pass@1**

Model	Baseline	LLMgen	Increase(%)
gemini-2.0-flash	58.97%	62.82%	+3.85%
gemini-2.0-flash-lite	57.69%	55.13%	-2.56%
gpt-40-2024-08-06	64.74%	66.03%	+1.29%
gpt-4o-mini-07-18	53.21%	50.00%	-3.21%

+4.49%, while *GPT-4o* and *GPT-4o-mini* showed smaller but still consistent improvements of +2.57% and +3.20%, respectively. These results demonstrated that enhancing specifications prior to generation effectively improved code quality. Importantly, the improvements held across both Gemini and GPT models, confirming that DeepSpec provided robust benefits regardless of the underlying model.

4.3 Comparison between Baseline and LLMgen

*Introduction of Comparison between Baseline and LLMgen*Additionally, since LLMs could generate enhanced specifications themselves, we compared direct code generation from the original specification with a variant where the LLM first produced an enhanced specification before code generation *LLMgen*. This approach tested whether self-generated specifications could replace structured enhancements such as those in DeepSpec.

*Results and Analysis*On *gemini-2.0-flash* and *gpt-4o-2024-08-06*, LLMgen achieved modest improvements over the Baseline, with gains of +3.85% and +1.29%, respectively. However, on *gemini-2.0-flash-lite* and *gpt-4o-mini-2024-07-18*, performance dropped below the Baseline by -2.56% and 3.21%. This inconsistency highlights the instability of relying on self-generated specifications. While they sometimes capture useful information, they are also prone to hallucinations and fabricated details that degrade functional correctness. Therefore, compared with the consistent gains achieved by DeepSpec,

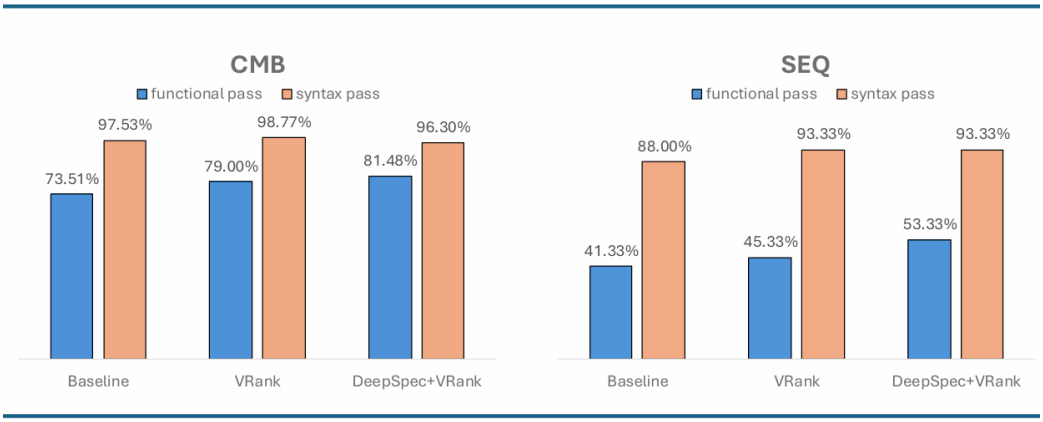


Figure 6: **Performance on circuit types for DeepSpec Framework.**

Table 3: **Performance of DeepSpec on a back-end framework.**

Model	**VRank**		**VRank+DeepSpec**		**Increase**
	Ratio	*#Tasks*	*Ratio*	*#Tasks*	*Ratio*
gemini-2.0-flash	62.82%	98/156	67.95%	106/156	+5.13%
gemini-2.0-flash-lite	57.10%	89/156	62.18%	97/156	+5.08%
gpt-4o-2024-08-06	66.67%	104/156	69.87%	109/156	+3.20%
gpt-4o-mini-2024-07-18	57.10%	89/156	61.53%	96/156	+4.43%

LLMgen remains unreliable and less effective.

4.4 Analysis by Circuit Type

In the previous section, we showed that DeepSpec improves the correctness of RTL designs. Here, we further compare three configurations: (i) using the original specification as a prompt to generate Verilog code, (ii) employing a back-end code framework (using VRank as an example) that focuses on code generation and selection, and (iii) combining the DeepSpec-enhanced specification with this back-end framework. We focus on combinational and sequential circuits, and the results are summarized in two bar charts (Fig. 2), which present the syntax and functional task pass rates under gemini-

2.0-flash. Within each chart, the pass rates are reported for the baseline, the back-end framework, and the combined workflow, enabling a direct comparison of DeepSpec's additive effect.

For combinational circuits, the baseline directly generating code with the original specification yields a lower functional accuracy of 73.51% (as shown in Table 3). Using the back-end code generation framework improves this to 79.00%, while further combining it with our DeepSpec framework achieves 81.48%. This combined approach first enhances the specification and then prompts code generation, highlighting that DeepSpec improves the reliability of functional logic while preserving high syntactic accuracy.

For sequential circuits, syntax correctness improves from 88.00% with the original specification to 93.33% when enhanced by our DeepSpec framework. Functional correctness shows the most notable improvement: it increases from 41.33% with the original specification to 45.33% with the VRank framework, and further to 53.33% when integrated with our DeepSpec framework. This demonstrates DeepSpec's contribution to handling temporal behaviors and state-dependent logic.

In summary, while syntax correctness is largely ensured by the back-end framework, DeepSpec provides substantial additive improvements in functional correctness. The integration of enhanced specifications with a back-end framework thus represents the most robust approach, outperforming either component used in isolation, validating the crucial role of DeepSpec in enhancing logical reasoning and ensuring functional consistency.

4.5 Ablation Study

Third, we conducted an ablation study under *gemini-2.0-flash* to evaluate the contribution of each component in our DeepSpec framework. Table 4 defines our metrics, where Eval0 measures syntax correctness but reflects

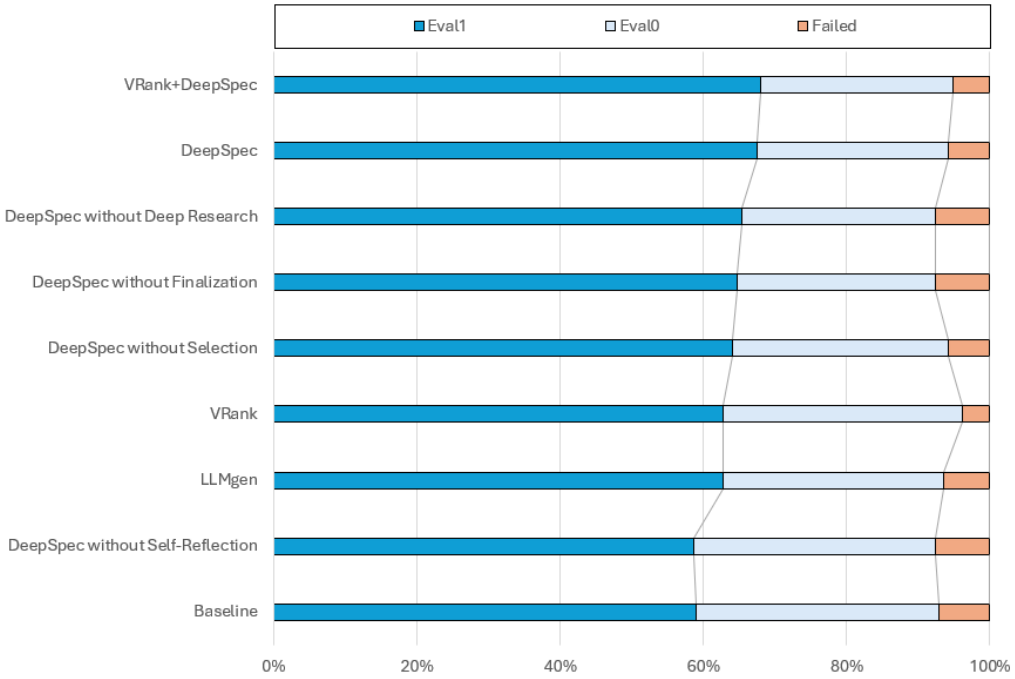


Figure 7: **Ablation Study for DeepSpec Framework.**

Table 4: **Definitions of Evaluation Criterion for ablation study.**

Type	Definition
Failed	RTL codes have syntax error
Eval0	RTL codes have no syntax error but have mismatches against golden Testbench
Eval1	RTL codes passed Eval0; report passed with the golden Testbench

cases where Verilog code is syntactically valid but functionally incorrect. and Eval1 measures functional correctness against golden testbenches.
The results are summarized as follows:

1. *Baseline:* Generates Verilog code directly from the original specification.

2. *LLMGen:* Generates Verilog code using the large language model's self-generated specification.

3. *VRank:* Uses the original specification to generate multiple code samples and select among them.

4. *DeepSpec without Selection:* Removes the Selection module, which performs high-level analysis of module header, logic, and strategy.

5. *DeepSpec without Deep Research:* Removes the Web Research module that collects external knowledge to support generation.

6. *DeepSpec without Reflection:* Removes the Reflection module that evaluates and refines the enhanced specification against the original one.

7. *DeepSpec without Finalization:* Removes the Finalization step that consolidates the enhanced specification and verifies logic/timing consistency.

8. *DeepSpec:* Full framework including Selection, Research, Reflection, and Finalization.

9. *DeepSpec + VRank:* Combines DeepSpec with multi-candidate generation and ranking.

The results are summarized in a bar chart Fig. 7. The Baseline shows the weakest performance, as direct generation without structured guidance often produces functionally incorrect designs. Adding VRank improves results by filtering weaker candidates, but the gains are limited compared with those achieved by DeepSpec. In contrast, DeepSpec substantially enhances functional correctness through its enhanced specifications, effectively narrowing the gap between syntax validity and functionality.

Within DeepSpec, Self-Reflection proves to be the most critical component: removing it causes performance to drop close to the baseline, showing that iterative refinement is essential. The Selection module also contributes significantly by performing top-level analysis of the original specification to guide design choices. The Deep Research module adds further improvements by introducing external knowledge, which reduces LLM hallucinations and prevents fabricated information, though its effect is smaller than Reflection and Selection. The Finalization module focuses on guiding circuit handling, reducing semantic ambiguity while performing logic analysis and processing

port feedback. In this way, it plays a stabilizing role that ensures workflow consistency.

Combining DeepSpec with VRank further strengthens syntactic robustness, but most of the functional accuracy gains come from DeepSpec itself. Overall, these findings confirm that DeepSpec is the decisive factor in transforming syntactically valid RTL into functionally correct designs.

5 Conclusion

In conclusion, we introduced DeepSpec, a workflow that significantly enhances RTL code generation. Our approach improves the specification process and can be flexibly integrated with other frameworks. We developed a circuit-type-aware strategy for detailed augmentation and leveraged deep research to effectively minimize hallucinations. Furthermore, a self-reflection mechanism rigorously verified the reliability of our specifications. During this process, a human-readable and actionable feedback report is provided for better human-machine collaboration in hardware design.

References

[1] K. Xu, G. L. Zhang, X. Yin, C. Zhuo, U. Schlichtmann, and B. Li. Automated c/c++ program repair for high-level synthesis via large language models. *arXiv preprint arXiv:2407.03889*, 2024. eess.

[2] Y. Li, D. Choi, J. Chung, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

[3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, others, and B. McGrew. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[4] F. Ui, C. Yin, K. Zhou, Y. Xiao, G. Sun, Q. Xu, and D. Lin. Origen: Enhancing rtl code generation with code-to-code augmentation and self-reflection. In *43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2024.

[5] M. Gao, J. Zhao, Z. Lin, W. Ding, X. Hou, Y. Feng, and M. Guo. Autovcoder: A systematic framework for automated verilog code generation using llms. In *2024 IEEE 42nd International Conference on Computer Design (ICCD)*, pages 162–169. IEEE, 2024.

[6] O. Cinquin. Chip-gpt: A managed large language model for robust data extraction from biomedical database records. *Briefings in Bioinformatics*, 25(2), 2024.

[7] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri. Autochip: Automating hdl generation using llm feedback. *arXiv preprint arXiv:2311.04887*, 2023.

[8] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li. Autobench: Automatic testbench generation and evaluation using llms for hdl design. In *2024 ACM/IEEE International Symposium on Machine Learning for CAD*, pages 1–10, 2024.

[9] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li. Correctbench: Automatic testbench generation with functional self-correction using llms for hdl design. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7. IEEE, 2025.

[10] T. H. A. Chi, C. Mackin, L. Shi, P. Vijayaraghavan, S. Tsai, and E. Degan. Rtlexplain: A structured approach to rtl code summarization and question answering for medium-to-large designs using llms. In *ACM/IEEE Design Automation Conference*, June 2025.

[11] M. Liu, N. Pinckney, B. Khailany, and H. Ren. Verilogeval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–8. IEEE, 2023.

[12] Y. Zhao, H. Zhang, H. Huang, Z. Yu, and J. Zhao. Mage: A multi-agent engine for automated rtl code generation. *arXiv preprint arXiv:2412.07822*, 2024.

[13] Z. Zhao, R. Qiu, C. Lin, G. L. Zhang, B. Li, and U. Schlichtmann. Vrank: Enhancing verilog code generation from large language models via self-consistency. In *2025 26th International Symposium on Quality Electronic Design (ISQED)*, pages 1–7. IEEE, 2025.

[14] R. Tang, D. Kong, L. Huang, and H. Xue. Large language models can be lazy learners: Analyze shortcuts in in-context learning. *arXiv preprint arXiv:2305.17256*, 2023.

[15] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, and Z. Zheng. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):481–503, 2025.

[16] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, and Y. Ma. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971*, 2024.

[17] Z. Chen, Z. Lin, X. Chen, Z. Liu, C. Liu, Y. Qiao, Y. Wang, and F. Yang. Spec2Doc2RTL: RTL Generation from Specification with Natural Language Representation. In *2025 International Symposium of Electronics Design Automation (ISEDA)*, pages 784–789. IEEE, 2025.

[18] Adam Kalai, Santosh Vempala, Ofir Nachum, Eddie Zhang, David Robinson, Saachi Jain, Eric Mitchell, Alex Beutel, and Johannes Heidecke. Why language models hallucinate. OpenAI Blog, September 2025.

[19] L. Langchain. Langgraph: Build resilient language agents as graphs. GitHub repository, 2024.

[20] Google. Gemini fullstack langgraph quickstart: Get started with building fullstack agents using gemini 2.5 and langgraph. GitHub repository, 2025.

[21] Gerard Salton, Anita Wong, and Chung S. Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620, 1975.

[22] Paul Jaccard. The distribution of the flora in the alpine zone. *New phytologist*, 11(2):37–50, 1912.

[23] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alethea Power Ray, Gretchen Krueger Puri, Roman Petrov, Tim Savinov, Igor Babuschkin Clarke, Miljan Plappert, Alexander Pavlov, Filip Chociej, Michael Litwin, Scott Gray, Nikolas Tezak, Felipe Petroski Such Tang, Igor Babuschkin, Jack Clark, Christopher Berner, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.